

Malnad College Of Engineering

Under the auspices of M.T.E.S[®]

(An Autonomous Institution Affiliated to VTU, Belgaum)

P.B. No. 21, Hassan 573 202, Karnataka



INFORMATION SCIENCE AND ENGINEERING

COURSE TITLE: FULLSTACK DEVELOPMENT(23IS553)

TOPIC : ONLINE QUIZ APP (MACROPROJECT)

Submitted By : Group 18

Name	USN
Harshitha H S	4MC23IS039
Harshitha N	4MC23IS041
Jeevitha D J	4MC23IS051
Prerana K	4MC23IS081
Monisha K P	4MC23IS064

Under The Guidance of:

Mr.Krishna Swaroop A,

Assistant Professor,

Department of ISE,MCE

2025-26

Submission Date:

28/11/2025

1. Abstract

The *Student Quiz Application* is a Django-based web platform designed to simplify quiz creation, student assessment, and result evaluation in educational institutions. The system enables teachers to create subjects, add quizzes, prepare multiple-choice questions, and evaluate student performance automatically. Students can register, log in, choose a subject, attempt quizzes, and instantly receive their results.

The application addresses the limitations of traditional classroom assessments, such as manual correction, paperwork, time consumption, and lack of performance tracking. By automating question handling, scoring, and record maintenance, the project ensures accuracy, efficiency, and improved accessibility.

Built using Django as the backend framework and supported by HTML, CSS, and Bootstrap on the frontend, the system uses SQLite3 for lightweight data storage. User authentication ensures secure access for both teachers and students. The platform provides a clean interface, structured workflow, and real-time score generation.

Overall, the application aims to digitalize academic assessments, reduce teacher workload, enhance student engagement, and enable a seamless learning–evaluation environment.

2. Introduction

In traditional education systems, conducting quizzes and tests involves significant manual effort from teachers, including question preparation, paper printing, evaluation, and result distribution. These processes are time-consuming, error-prone, and lack transparency. With the increasing adoption of digital learning platforms, there is a need for a reliable online assessment tool that supports quick evaluation and student performance tracking.

The *Student Quiz Application* was developed to address these needs using Django’s powerful Model–View–Template architecture. The domain was chosen because online assessments have become essential in schools, colleges, and competitive training environments. The project replicates real-world examination activities by allowing teachers to create structured quizzes and enabling students to attempt them remotely.

Existing systems often lack automation, require third-party tools, or do not provide customizable control to educators. This project offers a centralized and low-cost solution.

By integrating secure login, user roles, automated scoring, and instant results, the application bridges the gap between traditional exams and modern digital evaluation methods.

3. Objectives of the Project

- a. To develop a user-friendly online platform for teachers and students to conduct quizzes.
- b. To allow secure authentication for teachers and students.
- c. To simplify quiz creation and automate question management.
- d. To enable students to attempt quizzes easily and view instant results.
- e. To reduce manual correction work and increase evaluation accuracy.
- f. To store all quiz-related data securely using a relational database.
- g. To provide a platform that improves learning outcomes and reduces administrative workload.

4. System Requirements

4.1 Software Requirements

Software	Version / Details
Python	3.x
Django	5.x
Database	SQLite3
IDE	VS Code / PyCharm
Frontend	HTML, CSS, Bootstrap
Others	Django Authentication, Django ORM

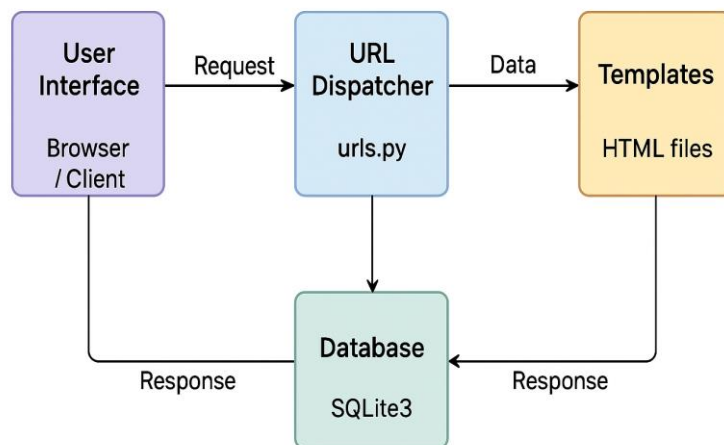
4.2 Hardware Requirements

- a. Minimum 4 GB RAM
- b. Intel i3 or above
- c. 500 MB free disk space

- d. Windows / Linux / macOS system

5. System Design

5.1 Architecture Diagram



5.2 Explanation of Architecture

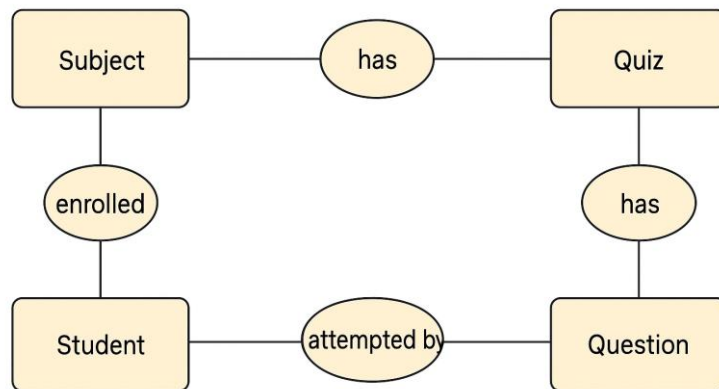
The system follows Django's **MVC/MVT Architecture**:

- User (Teacher/Student)** sends a request through browser.
- URL Dispatcher** forwards to appropriate view.
- Views** contain the logic for handling requests, quiz creation, quiz attempt, login, logout, etc.
- Models** interact with the database to store subjects, quizzes, questions, options, students, and results.
- Templates** display HTML pages rendered using context data.
- Database** stores all persistent information like users, quizzes, answers, and scores.

Data flows from user → URL → view → model → view → template → user.

6. Database Design

6.1 ER Diagram / Schema



6.2 Tables and Explanation

Table	Description
Profile	Stores whether a user is a teacher or student + basic info
Subject	Contains subject names (Maths, Science, etc.)
Quiz	Stores quiz title, subject, total questions
Question	Contains the question text + related quiz
Option	Stores 4 options for each question and marks the correct one
Result	Stores student score for each quiz

6.3 Each table has **primary keys** and uses **foreign keys** to maintain relationships.

1. Profile (Student/Teacher Profile)

The Profile model stores additional information about each user that is not included in the default Django User model. It defines whether a user is a *student* or a *teacher* through fields like `is_student` and `is_teacher`. For students, the profile may include details such as age,

standard, or other academic information. For teachers, it identifies their role in managing subjects, quizzes, and questions. This separation helps maintain clean user authentication while adding custom data based on user type, allowing the application to differentiate user permissions and dashboards.

2. Subject

The Subject model represents the academic subjects offered within the quiz system, such as Mathematics, Science, English, etc. It acts as the top-level category under which quizzes are created. Each subject can have multiple quizzes, and students may attempt quizzes based on the subjects assigned by the teacher. The Subject model keeps the platform organized and allows teachers to manage content efficiently. It also helps in grouping quizzes, filtering student activities, and generating performance reports based on specific subjects.

3. Quiz

The Quiz model represents a set of questions related to a particular subject. Each quiz belongs to a specific subject and is created by a teacher. The model contains details like quiz title, creation date, and instructions if needed. A quiz can have multiple questions, and students must attempt all of them to receive a final score. When a student submits answers, the quiz model works with the Result model to calculate marks. The Quiz model plays a central role in managing assessments and tracking student learning progress.

4. Question

The Question model stores each individual quiz question. Every question belongs to a particular quiz and includes four options for students to choose from. It contains the question text and the correct answer for evaluation. The model ensures that teachers can add unlimited questions to any quiz while maintaining structure and consistency. During an attempt, questions are displayed one after another or together, depending on the design. When the student submits answers, the system checks correctness based on this model. The Question model ensures accurate assessment and is essential for generating scores in the Result model.

5. Option

The Option model represents the available answer choices for each question. Each question has four options, and each option contains text along with a flag (`is_correct`) to indicate the right answer. This structure allows the system to handle multiple-choice questions efficiently.

Instead of storing all choices in the question model, separating them into a dedicated Option model makes the system cleaner, more scalable, and easier to manage. It also enables flexible modification of options by teachers when editing or updating quiz content.

6. Result

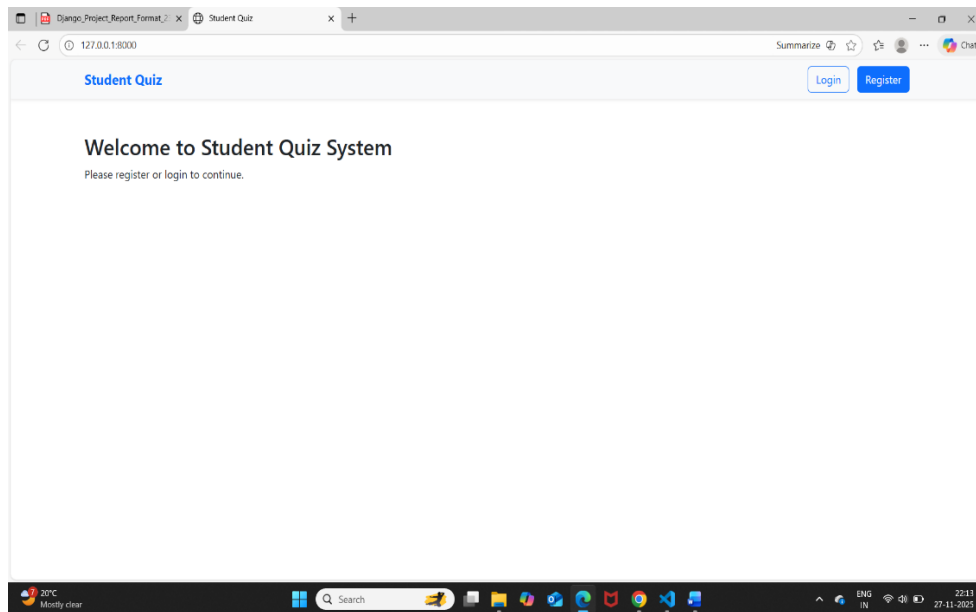
The Result model stores the outcome of a student's quiz attempt. It records details such as the student who attempted the quiz, the quiz title, score obtained, total marks, and submission time. After the student submits answers, the system evaluates correctness and saves the result automatically. This model is crucial for generating performance reports and allows students to view past attempts. Teachers can use the Result model to track student progress, identify weak areas, and analyze performance across different subjects. It forms the analytical backbone of the entire quiz system.

7. Implementation (High-level)

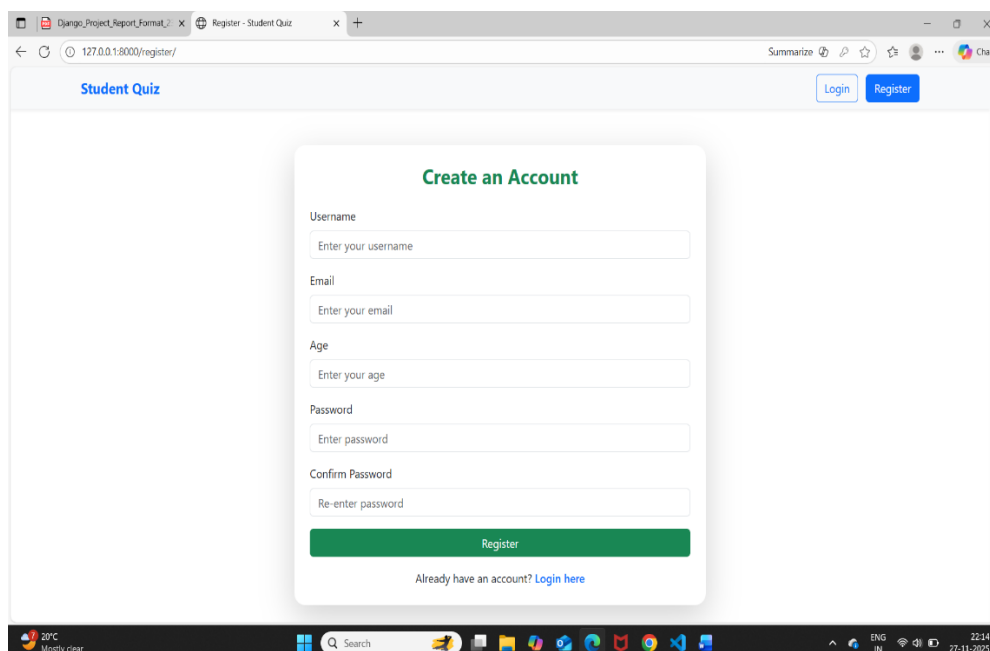
- a. The system uses Django models to define database structure.
- b. Views are created for quiz creation, question addition, student quiz attempt, and result submission.
- c. URL routing maps each feature to appropriate view.
- d. HTML templates use Bootstrap for clean UI.
- e. Django authentication handles login, logout, and signup.
- f. Quiz scoring is automated by comparing selected options with correct answers.

8. Screenshots

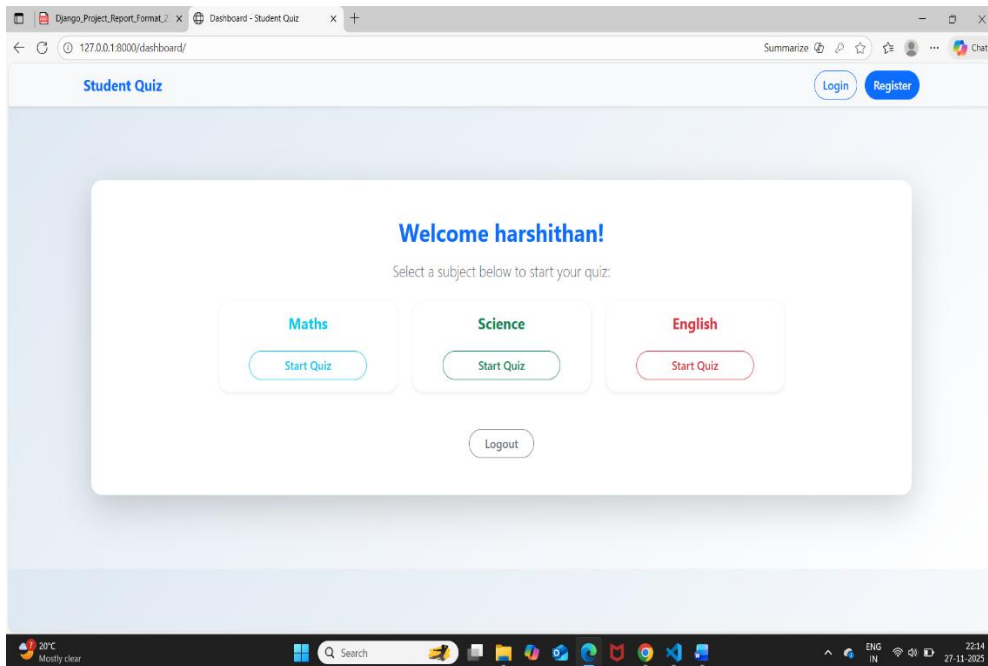
8.1 Homepage



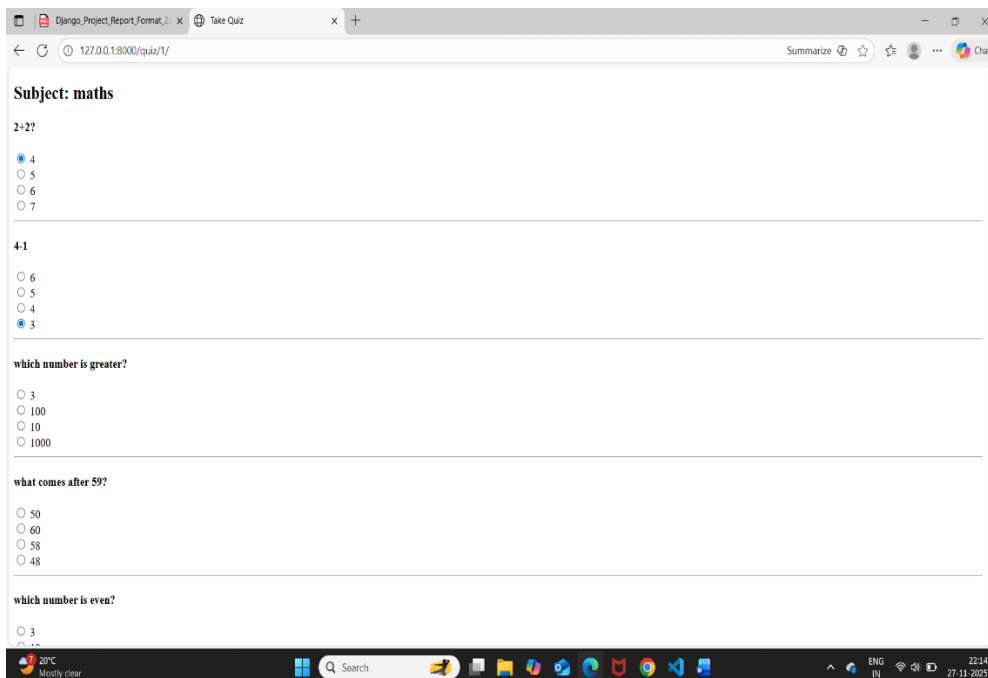
8.2 Login/Register Pages



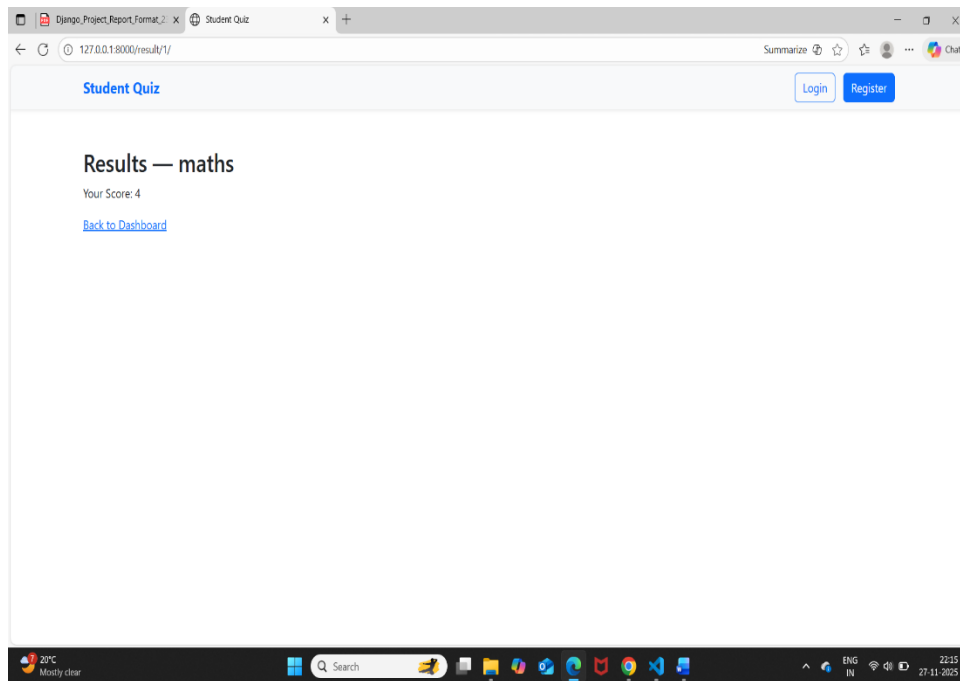
8.3 Student Interest Page



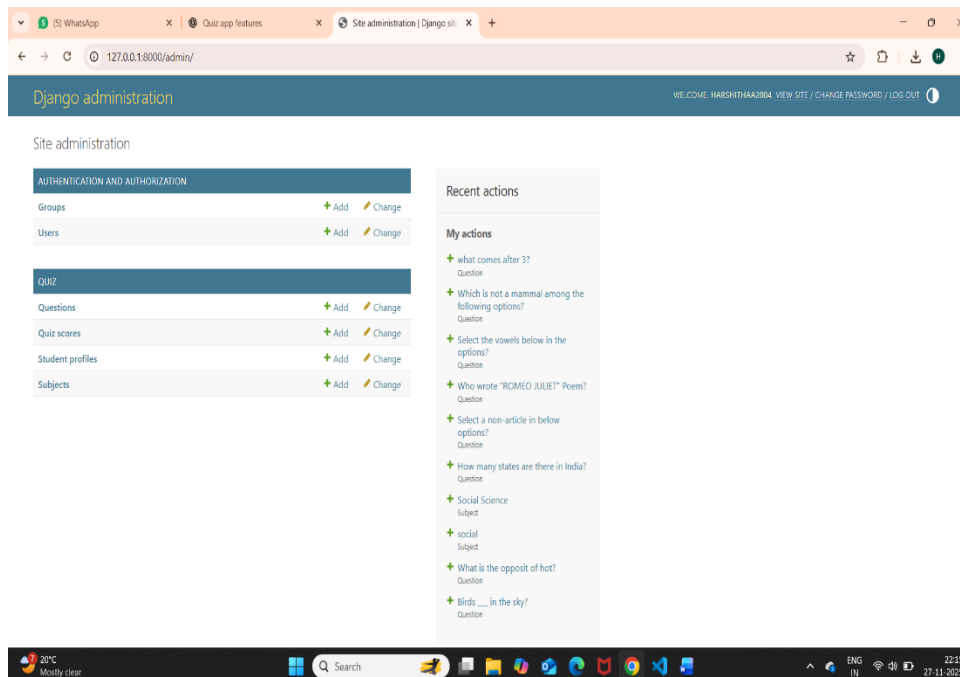
8.4 Quiz Page



8.5 Score Display Page



8.6 Admin Panel



9. Testing

Test Case	Expected Output
Login with wrong password	Error message shown
Register without required field	Form validation error
Teacher adds quiz	Quiz saved in database
Student submits quiz	Score is generated
Logout	Redirects to login page

10. Results

The *Student Quiz Application* successfully automates the entire quiz management system. Teachers can efficiently create quizzes and manage student results, while students can take quizzes and access results instantly. The system reduces manual effort, increases accuracy, and provides a structured evaluation environment.

11. Conclusion

The project achieves its objective of building an efficient online quiz platform with secure user authentication, quiz automation, and instant result generation. By digitalizing the evaluation process, the system reduces teacher workload and enhances student learning. It is scalable and can be extended with more features in future.

12. Future Enhancements

- a. Add timer-based quizzes
- b. Export results to PDF / Excel
- c. Add analytics dashboard for teachers
- d. Live quiz competitions

- e. Mobile app version

13. References

- a. Django Documentation
- b. W3Schools HTML/CSS
- c. Bootstrap Documentation
- d. Stack Overflow Discussions

